Solution for Assignment 01 Question 3 part c

In [1]:
```
# Solution for Assignment 01 Q3 part c

# Imports and seed setup
import torch

# Constants
w_init = torch.tensor(-0.9)                  # Starting weight
lr = 1 / (2 * torch.sqrt(torch.tensor(3.0))) # Learning Rate eta
w_bar = torch.tensor(1)                      # w_bar parameter value

# Loss(w) = w(w^2 / 3 - w_bar^2)
def Loss(weights, w_bar_loss):
    return weights * (weights ** 2 / 3 - w_bar_loss ** 2)

# Gradient of the loss = w^2 - w_bar^2
def Grad_Loss(weights, w_bar_loss):
    return weights ** 2 - w_bar_loss ** 2

# PI returns the updated weights clamped to be between -sqrt(3) to sqrt(3)
def PI(weights, eta, gradient_loss, w_bar_loss):
    new_weights = weights - eta * gradient_loss(weights, w_bar_loss)
    new_weights = torch.clamp(
        new_weights,
        min=-1 * torch.sqrt(torch.tensor(3)),
        max=torch.sqrt(torch.tensor(3)),
    )
    return new_weights

def PGD(w_init, w_bar_loss, eta, update_function, loss_function, gradient_loss):
    # Initialize the starting loss to infinity
    loss_val = torch.tensor(float("inf"))
    iters_to_converge = 0
    weights = torch.clone(w_init)
    # Keep running until L(w_t) and L(w_{t+1}) are almost equal
    while not torch.isclose(loss_val, loss_function(weights, w_bar_loss)):
        loss_val = loss_function(weights, w_bar_loss)
        weights = update_function(weights, eta, gradient_loss, w_bar_loss)
        iters_to_converge += 1
    return weights, iters_to_converge

w_final, iters_to_converge = PGD(w_init, w_bar, lr, PI, Loss, Grad_Loss)

print(f'Converged to w: {w_final}')
print(f'Took: {iters_to_converge} iterations')
```

```
Converged to w: 0.9992519617080688
Took: 16 iterations
```

So the **PGD** did converge to the approximately optimal value w^*:

$$w_{\text{final}} = 0.9992519617 \approx 1 = w^* = \bar{w},$$

and took 16 iterations to converge (with the default tolerance on the loss difference).

In [2]:
```
w_init = torch.tensor(-1.1)  # Starting weight

w_final, iters_to_converge = PGD(w_init, w_bar, lr, PI, Loss, Grad_Loss)
```

```
print(f"Converged to w: {w_final}")
print(f"Took: {iters_to_converge} iterations")

w_init = torch.tensor(-1.0)  # Starting weight

w_final, iters_to_converge = PGD(w_init, w_bar, lr, PI, Loss, Grad_Loss)

print(f"Converged to w: {w_final}")
print(f"Took: {iters_to_converge} iterations")
```

```
Converged to w: -1.7320507764816284
Took: 5 iterations
Converged to w: -1.0
Took: 1 iterations
```

Neither PGD converged to w^*.

In the first case, with $w_1 = -1.1$, PGD took 5 iterations to converge to approximately $w_{\text{final}} \approx -\sqrt{3}$, i.e. it got stuck at the boundary of the feasible region defined by the projection.

In the second case, with $w_1 = -1.0$, PGD the derivative of the loss at the initial point is zero, so it converged in a single iteration to $w_{\text{final}} = -1.0$.